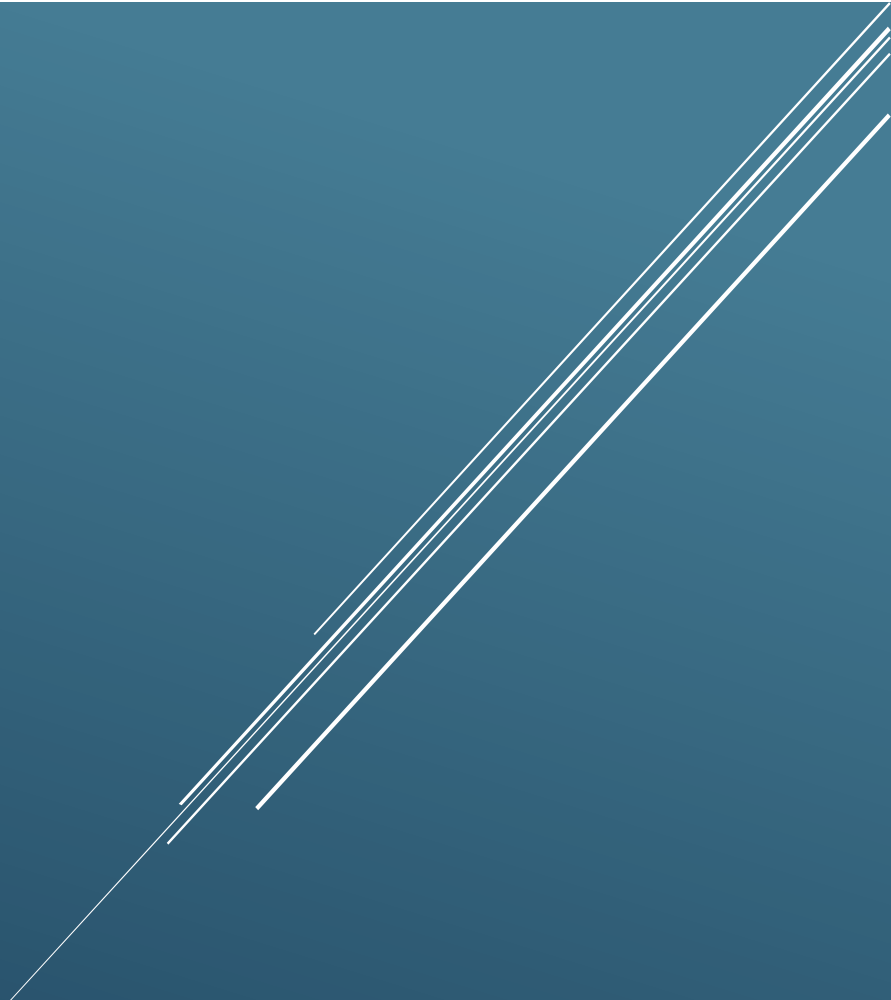




LINUX ROUTER CONFIGURATION

Technical report(202026364)

Introduction

This project explores that concept by configuring an Ubuntu Linux virtual machine to function as a fully operational router within a controlled lab environment.

Using VirtualBox as the virtualization platform, the Ubuntu machine was positioned between a simulated local network (LAN) and an external network representing internet access (WAN). Several networking technologies were put into practice to make this work: Network Address Translation (NAT) allows multiple LAN devices to share a single public-facing IP address, a DHCP server was deployed to automatically assign IP addresses to client machines, firewall rules were written using iptables to manage and secure traffic flow, and IP forwarding was enabled in the kernel to allow packets to travel between the two interfaces. Together, these components replicate the core behaviour of a real-world router.

Objectives

The primary goal of this project was to show that a standard Linux system, given the right configuration, can serve as a capable and functional router. Rather than treating networking as an abstract subject, this hands-on approach required actively setting up each component and verifying that it worked correctly. The specific objectives pursued throughout the project were:

- Enable IP forwarding on the Linux kernel to allow packets to travel between network interfaces.
- Configure Network Address Translation (NAT) using iptables so that LAN clients could reach the internet through the router.
- Deploy and configure a DHCP server to automatically distribute IP addresses to devices on the local network.
- Write and persist firewall rules to control which traffic is permitted to pass through the router.
- Establish proper routing between the LAN (internal) and WAN (external) interfaces so that end-to-end communication was achievable.

System Setup

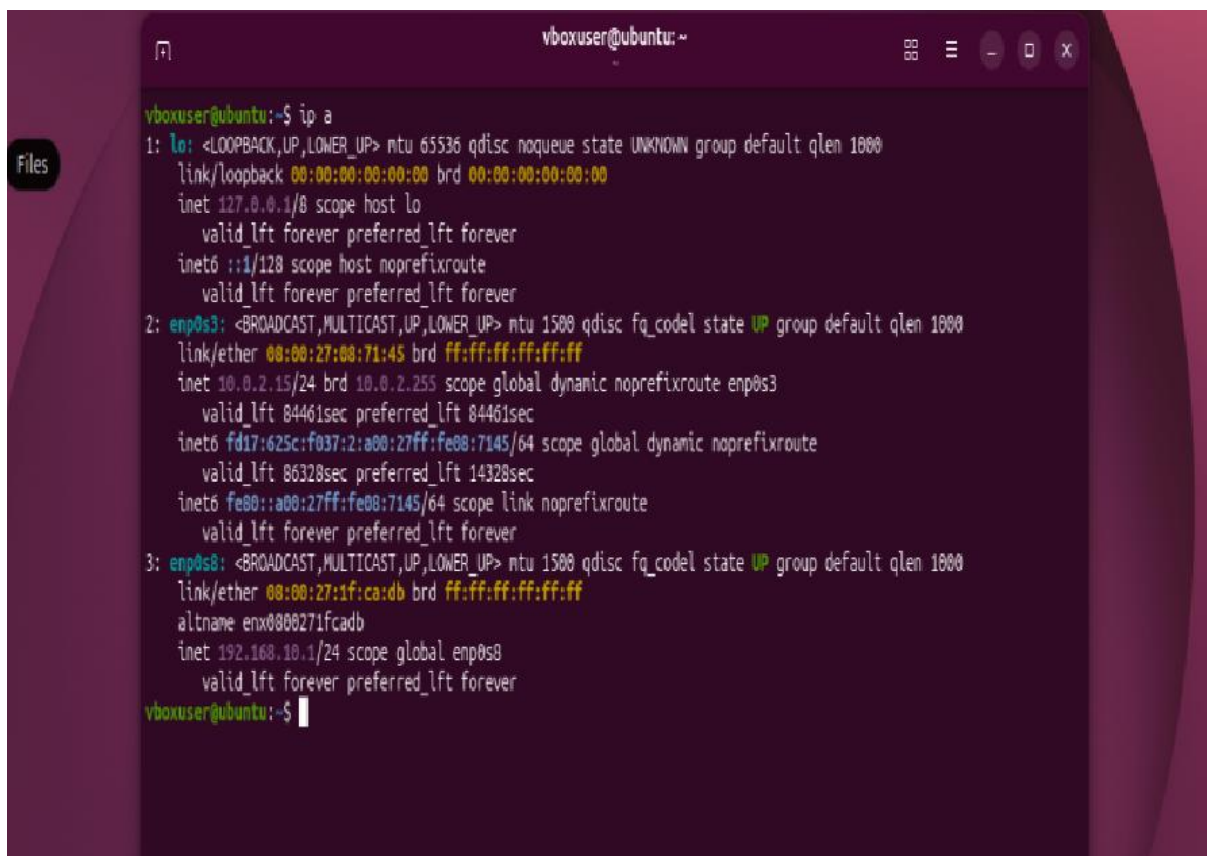
Virtualization Environment:

The entire lab was built inside VirtualBox, which provided a safe and isolated space to experiment with network configurations without affecting any physical infrastructure. Two virtual machines were created: an Ubuntu machine to serve as the router, and a Kali Linux machine to act as a client device on the local network. This arrangement mirrors a typical home or small-office setup, where a router sits between client devices and the internet, handling traffic on behalf of everyone on the inside.

Network Interfaces:

➤ Ubuntu:

- enp0s3 (NAT - Internet)
- enp0s8 (Internal Network - LAN)

A screenshot of a terminal window titled 'vboxuser@ubuntu: ~'. The terminal shows the output of the 'ip a' command. It lists three network interfaces: 'lo' (loopback), 'enp0s3' (NAT - Internet), and 'enp0s8' (Internal Network - LAN). The 'enp0s3' interface is configured with IP 10.0.2.15 and MAC address 08:00:27:08:71:45. The 'enp0s8' interface is configured with IP 192.168.10.1 and MAC address 08:00:27:1f:ca:db. The terminal output is as follows:

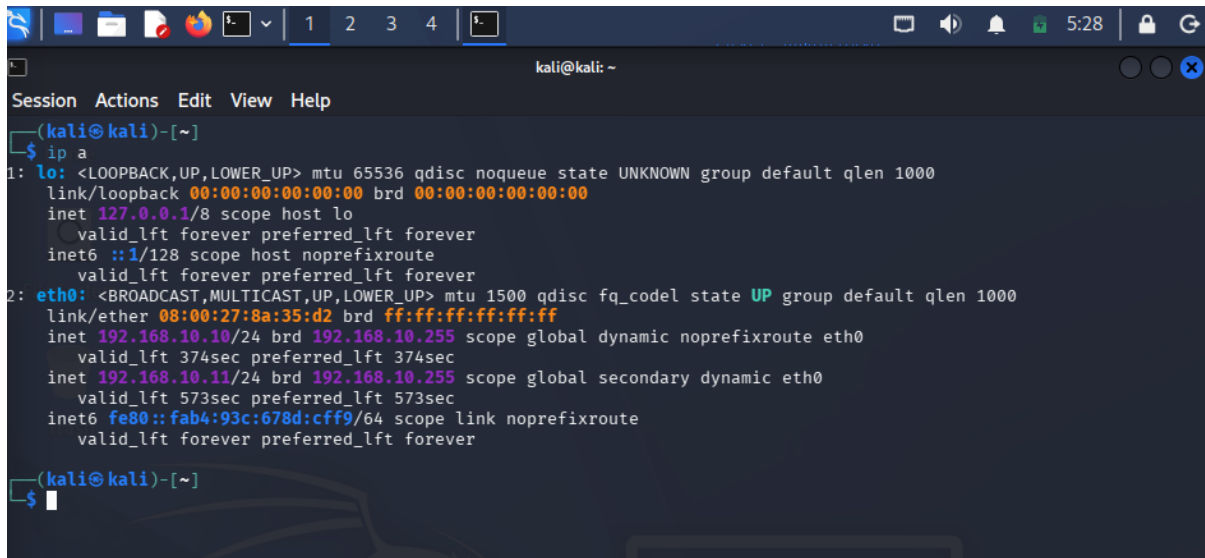
```
vboxuser@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:08:71:45 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 84461sec preferred_lft 84461sec
    inet6 fd17:625c:f037:2:a00:27ff:fe08:7145/64 scope global dynamic noprefixroute
        valid_lft 86328sec preferred_lft 14328sec
    inet6 fe80::a00:27ff:fe08:7145/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:1f:ca:db brd ff:ff:ff:ff:ff:ff
    altname enx0800271fcadb
    inet 192.168.10.1/24 scope global enp0s8
        valid_lft forever preferred_lft forever
vboxuser@ubuntu:~$
```

The screenshot above displays the network interface configuration on the Ubuntu router. The machine was given two virtual network adapters in VirtualBox to represent its two distinct roles. One adapter, enp0s3, was attached to a NAT network to provide access to the external internet, while the second adapter, enp0s8, was connected to an internal-only network representing the LAN side where client

machines would connect. Seeing both interfaces active and correctly assigned confirmed that the virtual hardware was in the right state before any software configuration began.

➤ Kali:

- eth0 (Internal Network)



```
(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.10/24 brd 192.168.10.255 scope global dynamic noprefixroute eth0
        valid_lft 374sec preferred_lft 374sec
    inet 192.168.10.11/24 brd 192.168.10.255 scope global secondary dynamic eth0
        valid_lft 573sec preferred_lft 573sec
    inet6 fe80::fab4:93c:678d:cff9/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$
```

The screenshot above shows the Kali machine shortly after connecting to the network. It was assigned the address 192.168.10.10, which falls squarely within the configured DHCP pool. The default gateway is correctly listed as 192.168.10.1, pointing back to the router, and the DNS servers match the values specified in the DHCP configuration. This result confirmed that the DHCP server was operating correctly and that the client had received all the information it needed to route traffic through the Ubuntu router.

Network Interface Summary

The following interfaces were used across the two virtual machines throughout the project:

- Ubuntu (Router): enp0s3 connected to the NAT/WAN network; enp0s8 connected to the internal LAN.
- Kali Linux (Client): eth0 connected to the same internal network as enp0s8 on the Ubuntu machine.

IP Addressing scheme

A dedicated subnet was planned for the local network to keep address management organised and predictable. The 192.168.10.0/24 range was selected for its simplicity

and its status as a commonly used private address space. The router was given the first usable host address in that range, 192.168.10.1, which would double as the default gateway for all client machines. DHCP was configured to assign addresses from 192.168.10.10 through to 192.168.10.200, leaving the lower addresses in the range available for any future static assignments.

Network Configuration

LAN Network: 192.168.10.0/24

Gateway (Router IP): 192.168.10.1

DHCP Range: 192.168.10.10 - 192.168.10.200

```
vboxuser@ubuntu: ~  
# be booted with DHCP, unless there is an address range on the subnet  
# to which a BOOTP client is connected which has the dynamic-bootp flag  
# set.  
#host fantasia {  
#   hardware ethernet 08:00:07:26:c0:a5;  
#   fixed-address fantasia.example.com;  
#}  
  
# You can declare a class of clients and then do address allocation  
# based on that.  The example below shows a case where all clients  
# in a certain class get addresses on the 10.17.224/24 subnet, and all  
# other clients get addresses on the 10.0.29/24 subnet.  
  
#class "foo" {  
#   match if substring (option vendor-class-identifier, 0, 4) = "SUNW";  
#}  
  
#shared-network 224-29 {  
#   subnet 10.17.224.0 netmask 255.255.255.0 {  
#     option routers rtr-224.example.org;  
#   }  
#   subnet 10.0.29.0 netmask 255.255.255.0 {  
#     option routers rtr-29.example.org;  
#   }  
#   pool {  
#     allow members of "foo";  
#     range 10.17.224.10 10.17.224.250;  
#   }  
#   pool {  
#     deny members of "foo";  
#     range 10.0.29.10 10.0.29.230;  
#   }  
#}  
subnet 192.168.10.0 netmask 255.255.255.0 {  
  range 192.168.10.10 192.168.10.200;  
  option routers 192.168.10.1;  
  option domain-name-servers 8.8.8.8, 1.1.1.1;  
}  
vboxuser@ubuntu: ~$
```

The screenshot above shows the DHCP configuration file after it was edited. The subnet block defines the address pool from which leases will be handed out, sets the default gateway clients should use, and specifies the DNS servers they should query. Google's public DNS server (8.8.8.8) and Cloudflare's resolver (1.1.1.1) were chosen to ensure reliable name resolution on the client side, independent of any network-specific proxy settings.

Ubuntu Router Configuration Steps

Before any routing could take place, the LAN-facing interface needed a fixed IP address. This is important because the router's LAN address becomes the default gateway for all client devices, so it must remain constant. If it changed, clients would lose their route to the outside world. The following commands were used to assign the address and bring the interface up:

1 Assign Static IP to LAN Interface

```
sudo ip addr add 192.168.10.1/24 dev enp0s8  
sudo ip link set enp0s8 up
```

These commands take effect immediately but do not persist through a reboot on their own. In a production environment, the address would typically be written into a Netplan or interfaces configuration file, but for this lab session the manual approach was sufficient.

2 Enable IP Forwarding

Temporary:

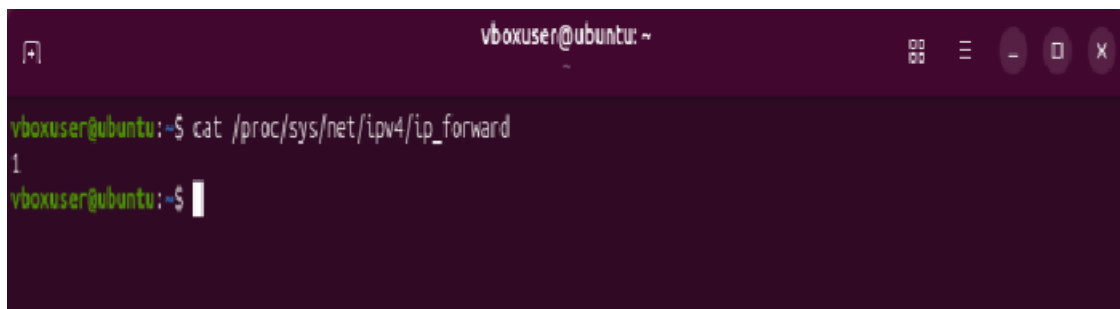
```
sudo sysctl -w net.ipv4.ip_forward=1
```

By default, Linux does not forward packets between network interfaces. This behaviour makes sense for ordinary workstations, where passing traffic from one network to another is neither expected nor desirable. For a router, however, forwarding packets between interfaces is the entire purpose of the machine, so the feature had to be explicitly switched on.

The fastest way to enable it is through the sysctl command, which takes effect immediately:

This change is temporary and will be lost after a reboot. To make it permanent, the `/etc/sysctl.conf` file was opened and the following line was added:

```
Edit /etc/sysctl.conf and set:  
net.ipv4.ip_forward=1
```

A terminal window with a dark purple background. The prompt is 'vboxuser@ubuntu: ~'. The command 'cat /proc/sys/net/ipv4/ip_forward' has been entered, and the output is '1'. The cursor is on a new line.

```
vboxuser@ubuntu: ~  
vboxuser@ubuntu:~$ cat /proc/sys/net/ipv4/ip_forward  
1  
vboxuser@ubuntu:~$
```

The screenshot above confirms that IP forwarding was successfully enabled on the router. The value returned by sysctl is 1, indicating that the kernel is now configured to pass packets from one interface to another. Without this step, all the NAT and firewall rules written later would have had no practical effect, because the kernel would have simply discarded any packet that arrived on one interface and was destined for a different network.

3 Configure NAT (iptables)

With IP forwarding active, the next step was to set up Network Address Translation. NAT allows the Kali client, which carries a private IP address, to communicate with the internet by having the router substitute its own WAN-facing IP address into outgoing packets. To an external server, all requests appear to come from the router rather than from individual clients. Three iptables rules were applied to achieve this:

```
sudo iptables -t nat -A POSTROUTING -o enp0s3 -j MASQUERADE  
sudo iptables -A FORWARD -i enp0s8 -o enp0s3 -j ACCEPT  
sudo iptables -A FORWARD -i enp0s3 -o enp0s8 -m state --state  
RELATED,ESTABLISHED -j ACCEPT
```

The first rule is the heart of NAT: it instructs the system to masquerade outgoing packets leaving through enp0s3, replacing the source address with the router's own external IP. The second rule explicitly permits forwarded traffic to flow from the LAN outward to the WAN. The third rule handles return traffic, only allowing packets back in from the internet if they belong to a connection that was already initiated from the inside. This final rule is an important security measure, as it ensures the router does not accept unsolicited inbound traffic while still allowing clients to receive replies to their own requests.

```
vboxuser@ubuntu: ~  
vboxuser@ubuntu:~$ sudo iptables -t nat -L -n -v  
Chain PREROUTING (policy ACCEPT 20 packets, 7704 bytes)  
pkts bytes target      prot opt in     out     source        destination  
  
Chain INPUT (policy ACCEPT 18 packets, 6552 bytes)  
pkts bytes target      prot opt in     out     source        destination  
  
Chain OUTPUT (policy ACCEPT 3406 packets, 375K bytes)  
pkts bytes target      prot opt in     out     source        destination  
  
Chain POSTROUTING (policy ACCEPT 1098 packets, 148K bytes)  
pkts bytes target      prot opt in     out     source        destination  
1508 227K MASQUERADE all -- *      enp0s3  0.0.0.0/0  0.0.0.0/0  
vboxuser@ubuntu:~$
```

The screenshot above shows the iptables rules as they appear once applied. The nat table confirms the MASQUERADE rule on the POSTROUTING chain, while the filter table shows both FORWARD rules controlling the direction and state conditions for traffic crossing the router.

4 Save Firewall Rules

One well-known limitation of iptables is that its rules exist only in memory. A system reboot clears the entire ruleset, which would mean reconfiguring the firewall manually after every restart. To avoid this, the iptables-persistent package was installed. This package provides a service that saves the current rules to disk and reloads them automatically whenever the system boots.

```
sudo apt install iptables-persistent -y  
sudo netfilter-persistent save
```

```
vboxuser@ubuntu: ~  
vboxuser@ubuntu:~$ sudo iptables -L -n -v  
Chain INPUT (policy ACCEPT 27337 packets, 19M bytes)  
pkts bytes target prot opt in out source destination  
  
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)  
pkts bytes target prot opt in out source destination  
0 0 ACCEPT all -- enp0s8 enp0s3 0.0.0.0/0 0.0.0.0/0  
0 0 ACCEPT all -- enp0s3 enp0s8 0.0.0.0/0 0.0.0.0/0 state RELATED,ESTABLISHED  
  
Chain OUTPUT (policy ACCEPT 17377 packets, 7979K bytes)  
pkts bytes target prot opt in out source destination  
vboxuser@ubuntu:~$
```

The screenshot above shows the firewall rules written to disk by the netfilter-persistent save command. The rules are stored in /etc/iptables/ and are automatically loaded by a systemd service during startup. From this point forward, the router's firewall configuration would survive reboots without any manual intervention.

5 Install and Configure DHCP Server

Rather than requiring every client to be configured with a manually assigned static IP address, a DHCP server was deployed on the router to handle address distribution automatically. This is standard practice in virtually every real-world network, from home routers to enterprise environments. The ISC DHCP server was selected as it is a mature, well-documented option for Debian-based Linux systems.

```
sudo apt install isc-dhcp-server -y
```

After installation, the server needed to be told which subnet it was serving, which addresses to lease out, and what network settings to provide to clients. The configuration added to /etc/dhcp/dhcpd.conf was:

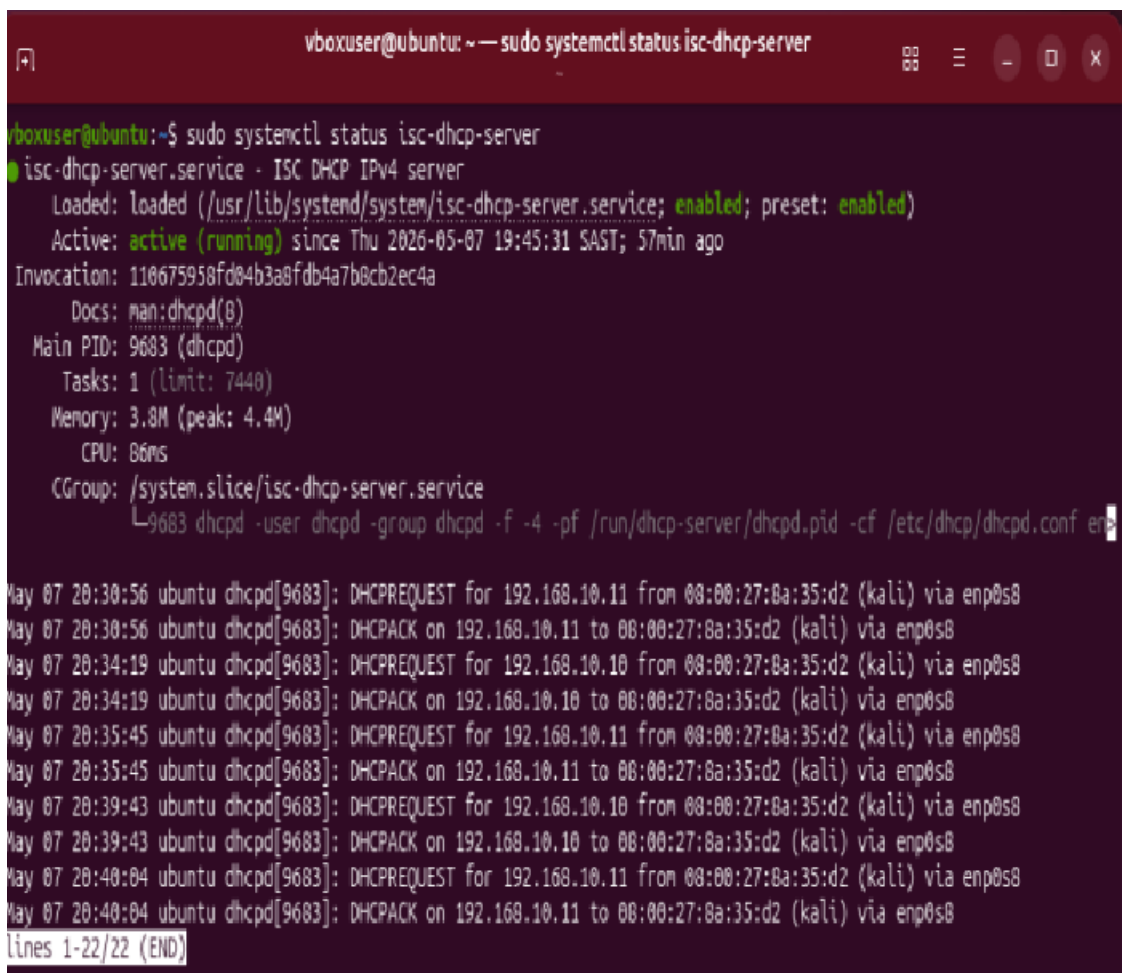
```
subnet 192.168.10.0 netmask 255.255.255.0 {  
    range 192.168.10.10 192.168.10.200;  
    option routers 192.168.10.1;  
    option domain-name-servers 8.8.8.8, 1.1.1.1;  
}
```

The server also required guidance on which interface to listen on. This was set in `/etc/default/isc-dhcp-server` by specifying the `INTERFACESv4` variable. Pointing it at `enp0s8` ensured the DHCP service only operated on the LAN side and would not attempt to issue leases on the external interface.

```
INTERFACESv4="enp0s8"
```

With the configuration complete, the service was started and enabled to launch automatically on boot:

```
sudo systemctl restart isc-dhcp-server
sudo systemctl enable isc-dhcp-server
```



```
vboxuser@ubuntu: ~ — sudo systemctl status isc-dhcp-server

vboxuser@ubuntu:~$ sudo systemctl status isc-dhcp-server
● isc-dhcp-server.service - ISC DHCP IPv4 server
   Loaded: loaded (/usr/lib/systemd/system/isc-dhcp-server.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-05-07 19:45:31 SAST; 57min ago
 Invocation: 110675958fd04b3a8fdb4a7b0cb2ec4a
   Docs: man:dhcpd(8)
  Main PID: 9683 (dhcpd)
    Tasks: 1 (limit: 7440)
   Memory: 3.8M (peak: 4.4M)
      CPU: 86ms
   CGroup: /system.slice/isc-dhcp-server.service
           └─9683 dhcpd -user dhcpd -group dhcpd -f -4 -pf /run/dhcp-server/dhcpd.pid -cf /etc/dhcp/dhcpd.conf etc

May 07 20:30:56 ubuntu dhcpd[9683]: DHCPREQUEST for 192.168.10.11 from 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:30:56 ubuntu dhcpd[9683]: DHCPACK on 192.168.10.11 to 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:34:19 ubuntu dhcpd[9683]: DHCPREQUEST for 192.168.10.10 from 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:34:19 ubuntu dhcpd[9683]: DHCPACK on 192.168.10.10 to 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:35:45 ubuntu dhcpd[9683]: DHCPREQUEST for 192.168.10.11 from 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:35:45 ubuntu dhcpd[9683]: DHCPACK on 192.168.10.11 to 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:39:43 ubuntu dhcpd[9683]: DHCPREQUEST for 192.168.10.10 from 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:39:43 ubuntu dhcpd[9683]: DHCPACK on 192.168.10.10 to 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:40:04 ubuntu dhcpd[9683]: DHCPREQUEST for 192.168.10.11 from 08:00:27:8a:35:d2 (kali) via enp0s8
May 07 20:40:04 ubuntu dhcpd[9683]: DHCPACK on 192.168.10.11 to 08:00:27:8a:35:d2 (kali) via enp0s8
lines 1-22/22 (END)
```

The screenshot above shows the DHCP server running with an active status. The `systemctl` output indicates that the service started cleanly and encountered no errors. At this point, any device connecting to the internal network would automatically

receive an IP address, default gateway, and DNS server configuration, with no manual setup required on the client side.

Kali Client Configuration

With the router fully set up, attention turned to the client machine to verify that everything was working correctly from the receiving end. The Kali Linux virtual machine was already attached to the same internal network as the router's enp0s8 interface. In principle, it only needed to request a DHCP lease and the rest should follow automatically.

Challenges Encountered

1. DNS resolution issues due to Eduroam proxy interference

After the Kali machine received an IP address and could successfully ping the router's LAN address, DNS resolution was still failing. Attempts to reach websites by hostname would time out, even though pinging external IP addresses like 8.8.8.8 worked fine. This pointed to a DNS-specific problem rather than a general routing issue. The cause was traced back to the university's Eduroam network, whose proxy configuration was overwriting the DNS entries in the Ubuntu machine's /etc/resolv.conf file. The fix involved manually replacing those entries with reliable public resolvers:

```
nameserver 8.8.8.8
nameserver 1.1.1.1
```

To prevent the system from silently overwriting these settings again, the file was locked using the `chattr` utility, which sets an immutable flag that prevents any process from modifying the file until the flag is explicitly removed:

```
sudo chattr +i /etc/resolv.conf
```

2. Missing dhclient on Kali

A separate issue was discovered when attempting to obtain a DHCP lease on the Kali machine: the `dhclient` tool, which is the standard utility for requesting leases on Linux, was not installed by default on that particular Kali image. Without it, the machine had no way to communicate with the DHCP server. Installing the missing package resolved the problem quickly, but the

experience was a reminder that tools which are assumed to be universally available sometimes need to be verified before troubleshooting begins.

3. Network interface confusion and packet loss

There were also a few moments of confusion when working with multiple network interfaces on the Ubuntu machine, particularly when both adapters were active and the traffic patterns were not immediately obvious. Careful use of the `ip addr` and `ip link` commands helped establish clarity on which interface was handling which type of traffic. Additionally, some intermittent packet loss was observed during testing, which appeared to stem from VirtualBox's internal networking rather than any misconfiguration. This resolved on its own and did not affect the final test results.

Testing and Results

Once all components were in place, a structured series of tests was carried out to confirm that the router was performing correctly from end to end. The tests moved progressively from the most basic level of connectivity upward to full internet access and name resolution.

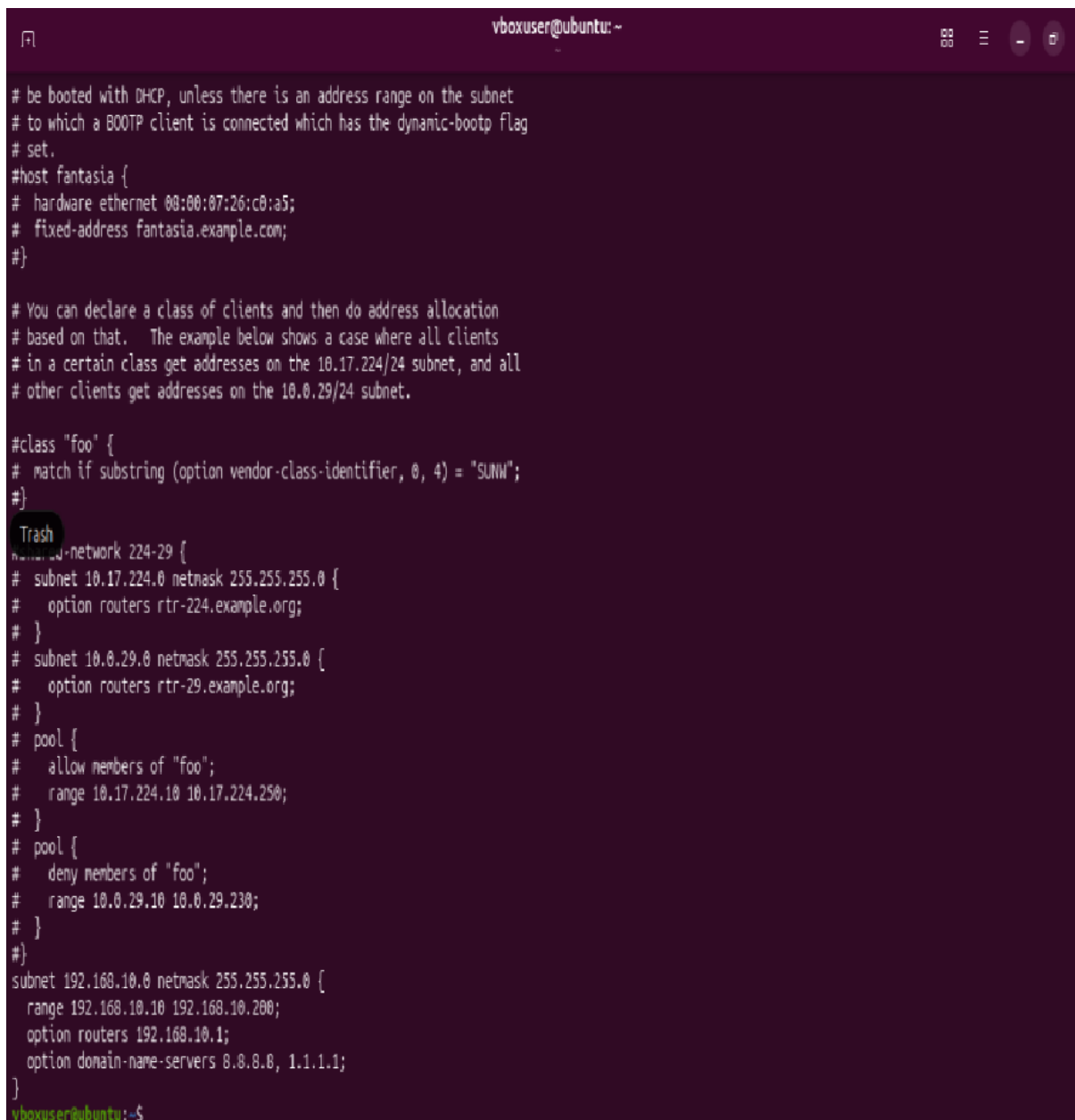
Tests performed:

The first test was a ping from the Kali client to the router's LAN IP address (192.168.10.1). A successful response here established that the client and router were talking correctly at the local network level. The next test was a ping to an external IP address (8.8.8.8), which would only succeed if the NAT rules were working and the router was properly forwarding outbound packets and returning the replies. The final test was a ping to google.com by name, which additionally required working DNS resolution. All three tests returned positive results.

```
kali@kali: ~  
Session Actions Edit View Help  
-(kali@kali)-[~]  
$ ping -c 5 192.168.10.10  
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data:  
bytes from 192.168.10.10: icmp_seq=1 ttl=64 time=0.025 ms  
bytes from 192.168.10.10: icmp_seq=2 ttl=64 time=0.165 ms  
bytes from 192.168.10.10: icmp_seq=3 ttl=64 time=0.099 ms  
bytes from 192.168.10.10: icmp_seq=4 ttl=64 time=0.057 ms  
bytes from 192.168.10.10: icmp_seq=5 ttl=64 time=0.068 ms  
  
- 192.168.10.10 ping statistics -  
packets transmitted, 5 received, 0% packet loss, time 4080ms  
rtt min/avg/max/mdev = 0.025/0.082/0.165/0.047 ms  
  
-(kali@kali)-[~]  
$ ping -c 5 8.8.8.8  
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:  
bytes from 8.8.8.8: icmp_seq=1 ttl=62 time=41.9 ms  
bytes from 8.8.8.8: icmp_seq=2 ttl=62 time=44.0 ms  
bytes from 8.8.8.8: icmp_seq=3 ttl=62 time=69.5 ms  
bytes from 8.8.8.8: icmp_seq=4 ttl=62 time=45.6 ms  
bytes from 8.8.8.8: icmp_seq=5 ttl=62 time=41.5 ms  
  
- 8.8.8.8 ping statistics -  
packets transmitted, 5 received, 0% packet loss, time 4038ms  
rtt min/avg/max/mdev = 41.451/48.503/69.513/10.610 ms  
  
-(kali@kali)-[~]  
$ ping -c 5 google.com  
PING google.com (142.251.47.46) 56(84) bytes of data:  
bytes from lcjnbi-aj-in-f14.1e100.net (142.251.47.46): icmp_seq=1 ttl=62 time=62.3 ms  
bytes from lcjnbi-aj-in-f14.1e100.net (142.251.47.46): icmp_seq=2 ttl=62 time=31.1 ms  
bytes from lcjnbi-aj-in-f14.1e100.net (142.251.47.46): icmp_seq=3 ttl=62 time=55.4 ms  
bytes from lcjnbi-aj-in-f14.1e100.net (142.251.47.46): icmp_seq=4 ttl=62 time=61.5 ms  
bytes from lcjnbi-aj-in-f14.1e100.net (142.251.47.46): icmp_seq=5 ttl=62 time=57.4 ms  
  
- google.com ping statistics -  
packets transmitted, 5 received, 0% packet loss, time 4080ms  
rtt min/avg/max/mdev = 31.082/53.540/62.347/11.516 ms  
  
-(kali@kali)-[~]  
$
```

The screenshot above captures the output of these three tests. Each ping completed successfully with low latency and no dropped packets. This confirmed that the entire network stack was functioning as intended: IP forwarding was moving packets between interfaces, NAT was translating addresses correctly on the way out and back, the DHCP server had equipped the client with the correct settings, and DNS was resolving hostnames without issue.

Configuration files

A screenshot of a terminal window with a dark purple background. The window title is 'vboxuser@ubuntu: ~'. The terminal displays a configuration file with various DHCP settings. Comments explain the purpose of different sections, such as host configuration and address allocation. The configuration includes a 'class' named 'foo' and two subnets: '10.17.224' and '10.0.29'. Each subnet has a netmask, a pool of addresses, and options for routers and domain-name-servers. The terminal ends with a prompt 'vboxuser@ubuntu:~\$'.

```
vboxuser@ubuntu: ~  
  
# be booted with DHCP, unless there is an address range on the subnet  
# to which a BOOTP client is connected which has the dynamic-bootp flag  
# set.  
#host fantasia {  
# hardware ethernet 08:00:07:26:c0:a5;  
# fixed-address fantasia.example.com;  
#}  
  
# You can declare a class of clients and then do address allocation  
# based on that. The example below shows a case where all clients  
# in a certain class get addresses on the 10.17.224/24 subnet, and all  
# other clients get addresses on the 10.0.29/24 subnet.  
  
#class "foo" {  
# match if substring (option vendor-class-identifier, 0, 4) = "SUNW";  
#}  
Trash  
#subnet 10.17.224 netmask 255.255.255.0 {  
# option routers rtr-224.example.org;  
# }  
#subnet 10.0.29 netmask 255.255.255.0 {  
# option routers rtr-29.example.org;  
# }  
# pool {  
# allow members of "foo";  
# range 10.17.224.10 10.17.224.250;  
# }  
# pool {  
# deny members of "foo";  
# range 10.0.29.10 10.0.29.230;  
# }  
#}  
subnet 192.168.10.0 netmask 255.255.255.0 {  
range 192.168.10.10 192.168.10.200;  
option routers 192.168.10.1;  
option domain-name-servers 8.8.8.8, 1.1.1.1;  
}  
vboxuser@ubuntu:~$
```

The screenshot above provides a view of the main configuration files that drive the router's behaviour. These include the DHCP server configuration in `dhcpd.conf`, the `sysctl.conf` file where IP forwarding is permanently enabled, and the iptables rules saved to disk by `iptables-persistent`. Reviewing these files served both as a record of what was configured and as a verification that all settings were correctly written to permanent storage rather than only held in memory.

vboxuser@ubuntu: ~ — sudo nano /etc/netplan/01-network-manager-all.yaml

GNU nano 8.7.1 /etc/netplan/01-network-manager-all.yaml

Let NetworkManager manage all devices on this system

network:

version: 2

renderer: NetworkManager

ethernet:

enp0s8:

dhcp4: no

addresses:

- 192.168.10.1/24

Read 9 lines

Show Apps

^O Write Out

^R Read File

^F Where Is

^X Replace

^K Cut

^V Paste

^T Execute

^J Justify

^C Location

^_ Go To Line

^U Undo

^E Redo

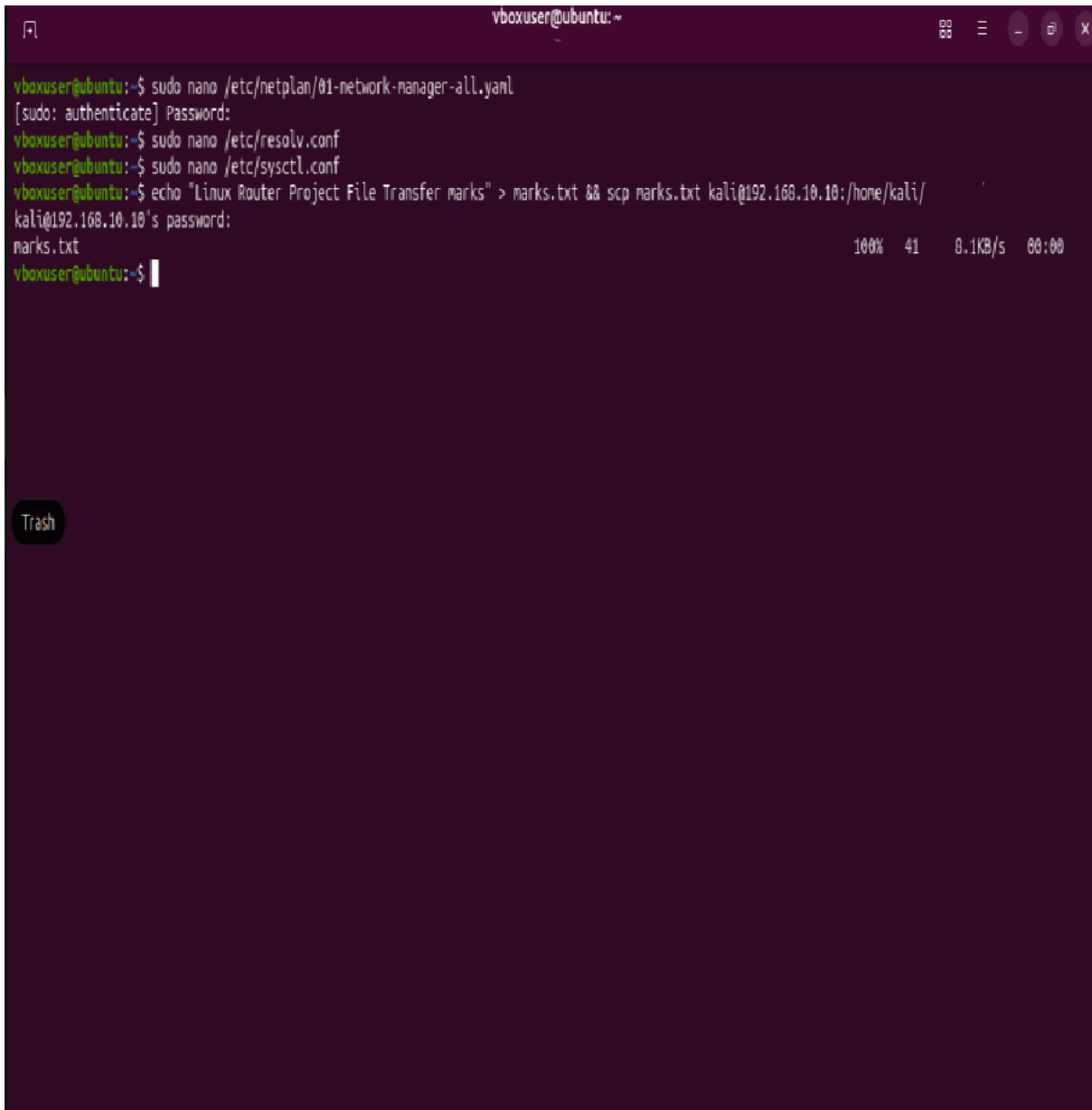
^M Set Mark

^D Copy

^I To Bracket

^B Where Was

File transfer

A terminal window titled 'vboxuser@ubuntu: ~' with standard window controls. The terminal shows a series of commands: 'sudo nano /etc/netplan/01-network-manager-all.yaml', '[sudo: authenticate] Password:', 'sudo nano /etc/resolv.conf', and 'sudo nano /etc/sysctl.conf'. Then, 'echo "Linux Router Project File Transfer marks" > marks.txt' is executed. Finally, 'scp marks.txt kali@192.168.10.10:/home/kali/' is run, followed by 'kali@192.168.10.10's password:' and 'marks.txt'. A progress bar shows '100%' completion, '41' files, and a speed of '0.1KB/s' in '00:00' time. The prompt returns to 'vboxuser@ubuntu:~\$'. A 'Trash' icon is visible in the bottom-left corner.

```
vboxuser@ubuntu:~$ sudo nano /etc/netplan/01-network-manager-all.yaml
[sudo: authenticate] Password:
vboxuser@ubuntu:~$ sudo nano /etc/resolv.conf
vboxuser@ubuntu:~$ sudo nano /etc/sysctl.conf
vboxuser@ubuntu:~$ echo "Linux Router Project File Transfer marks" > marks.txt
kali@192.168.10.10's password:
marks.txt                                     100% 41    0.1KB/s  00:00
vboxuser@ubuntu:~$
```

A file transfer was performed across the network to confirm that the router could handle continuous data traffic, not just isolated ping packets. A single ping verifies basic reachability, but sustained transfers exercise the NAT table, the forwarding chain, and the network stack more thoroughly. The transfer completed without errors, demonstrating that the router was capable of handling real-world workloads reliably.

Conclusion

This project demonstrated in a practical and tangible way that a standard Linux machine can be configured to carry out all the core functions of a dedicated network router. By working through each component in sequence, from enabling kernel-level IP forwarding and writing NAT rules to deploying a DHCP server and managing firewall policies, a fully operational routing environment was assembled entirely through software configuration.

The process also brought to light several real-world complications that are easy to overlook in purely theoretical study. DNS conflicts introduced by an external university proxy, missing tools on the client machine, and occasional packet loss from the virtualization layer all required practical problem-solving skills to resolve. Working through these issues deepened the understanding of how each component depends on and interacts with the others.

In the end, the Linux router passed all connectivity tests and demonstrated stable performance within the lab environment. The configuration provides a solid foundation that could be extended in future work to include more advanced networking features such as VLAN segmentation, traffic shaping, or VPN tunneling.

Recommendations

Looking back at this project as a whole, it is clear that working through a hands-on networking task like this one produces a very different kind of understanding compared to reading about the same concepts in a textbook. Before starting, terms like NAT, IP forwarding, and iptables were familiar at a surface level, but the actual mechanics of how they interact were not fully clear. Having to configure each component individually, and then troubleshoot when something did not behave as expected, made those relationships much more concrete.

One of the most valuable things learned was how tightly dependent each layer of the setup is on the ones beneath it. Enabling IP forwarding without NAT rules means packets go nowhere useful. NAT rules without a working DHCP lease on the client side means there is nothing to translate. Getting DNS wrong means everything looks broken even when the routing itself is fine. Understanding this chain of dependencies, and being able to isolate which layer was failing during troubleshooting, felt like a significant step forward in practical networking knowledge.

The challenges encountered during the project were arguably more instructive than the parts that went smoothly. The DNS issue caused by the Eduroam proxy, for instance, was not something that would appear in a standard lab guide. Diagnosing it required checking each component individually, ruling out routing as the cause, and eventually tracing the problem back to the `/etc/resolv.conf` file being silently overwritten. That kind of real-world interference is exactly what makes live lab work

valuable, and it built confidence in approaching unfamiliar networking problems methodically rather than randomly trying fixes.

Going forward, this project has sparked an interest in exploring more advanced routing concepts. Topics such as dynamic routing protocols, traffic filtering based on application layer data, and VPN configuration all feel more approachable now that the foundational mechanics are understood from first principles. The experience has also reinforced the importance of documentation: keeping track of which commands were run, in which order, and what each one was meant to achieve made it significantly easier to retrace steps when something went wrong.